Black-Box Adversarial Attack on a CNN

Problem 2 — decision-based (hard-label) attack on a pretrained CIFAR-10 CNN. 1000 clean images, one adversarial PNG each.

1. Results

metric	value
attack success rate	100.00 % (1000/1000)
L2 per image (0–255 scale)	mean 59.01 median 58.34 max 205.26
L∞ per image (0–255 scale)	mean 6.91 median 7 max 26
changed pixel values (of 3072)	mean 1,489 (48.5 %)
RMSE (0–255 scale)	mean 1.065
PSNR	mean 49.00 dB median 47.68 dB
SSIM	mean 0.99762 min 0.98323
model inferences	16,899,931 total 16,900 per image

All distances are measured between the clean PNG and the submitted adversarial PNG, on the integer 0–255 pixel scale, and the success rate is measured by re-reading the submitted files from disk. A median L2 of about 58 over 3072 values is a root-mean-square change of 1.1 pixel levels, i.e. 49 dB PSNR.

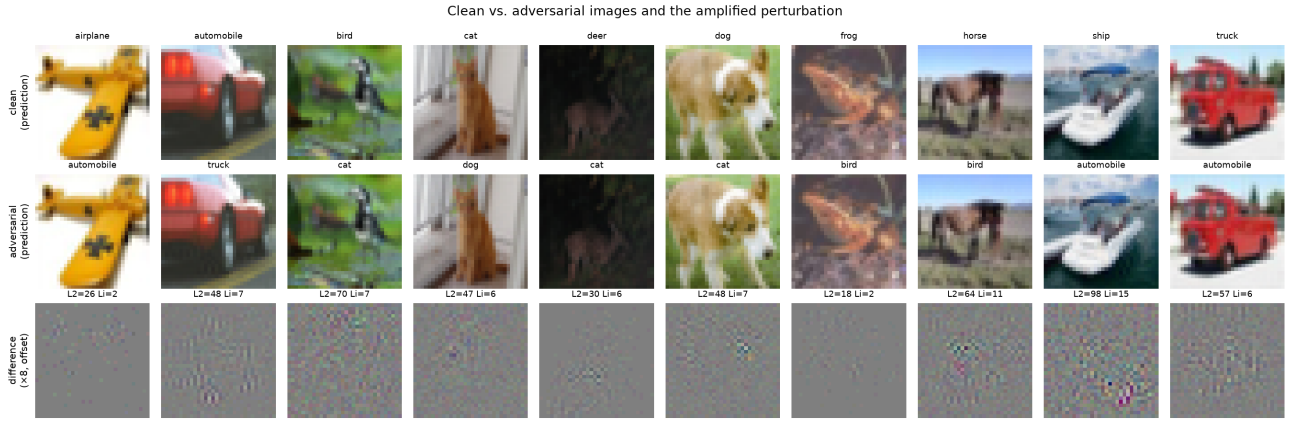


Figure 1. One image per class: clean (with the model's prediction), adversarial (with the model's new prediction), and the perturbation amplified 8× about mid-grey.

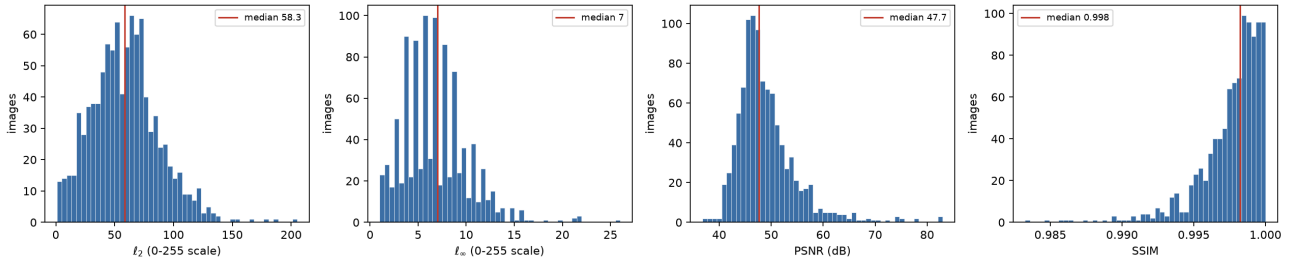


Figure 2. Distribution over the 1000 images of L2, L∞, PSNR and SSIM.

2. Threat model, and how it is enforced

The attacker may query the CNN at arbitrary inputs and observe the **output label**; architecture, parameters and all intermediate outputs (including class scores) are off limits. In the submitted code the model is reachable only through a single wrapper object whose entire public surface is `predict(x) → labels` and `is_adversarial(x, y) → bool`. Every stage of the attack consumes only those booleans, so no score can leak into a decision. The wrapper also counts each forward pass, which is where the inference numbers above come from.

Two properties of the oracle were verified before attacking, because the whole method depends on them: the predictions are deterministic, and they do not depend on how images are grouped into batches (a model left in training mode would use batch statistics and break both). Both checks are asserted at the start of every run.

3. Method

The attack minimises the perturbation norm subject to the label actually changing, driven only by the answer to "did the label change?". It is organised as a sequence of stages, each cheaper per unit of progress than the previous one is at that point.

Stage 0 — free adversarial seeds (0 queries)

The victim classifies all 1000 provided images correctly. Any clean image carrying a different label is therefore *already* an adversarial example for a given image, and the nearest such images in pixel space are strong starting points that cost nothing. This replaces the usual "draw random noise until it is misclassified" initialisation. The nearest seed is also a valid 8-bit PNG, so recording it at the outset places a **100 % success-rate floor** under the run: every later stage can only substitute something closer that the oracle has itself confirmed.

Stage 1 — bisection onto the decision boundary

Binary search along the segment from the clean image to a seed returns the smallest blend that is still misclassified. Four seeds are searched and the closest result is kept.

Stage 2 — HopSkipJump

The direction of the boundary normal is estimated by Monte-Carlo from the ±1 answers of random probes around the current boundary point; the point is then advanced along that estimate with a geometrically decreasing step and re-projected onto the boundary (Chen, Jordan & Wainwright, IEEE S&P 2020). Two details mattered more than the rest:

• **The re-projection must be fine.** The probe radius is proportional to `dist/d` with `d = 3072`, so it is tiny. If the re-projected point sits deeper inside the adversarial region than that radius, nearly every probe answers "still adversarial", the estimator saturates and progress stalls. Measured probe balance went from 0.75 (saturated) to 0.40 (healthy) when the bisection was deepened from 10 to 20 steps, and the median L2 at a fixed budget fell from 210 to 130.

• **Low-frequency probing hurt.** Restricting probes to a smooth upsampled subspace is a standard query-efficiency trick (QEBA), but this model needs high-frequency perturbations: at an equal budget the median L2 was 141 in full space against 165, 247 and 563 for 3·16·16, 3·8·8 and 3·4·4 subspaces. Full-space probing is used throughout.

Stage 3 — boundary refinement

Near convergence a fresh normal estimate costs hundreds of queries for a small gain, so the attack switches to two much cheaper moves, applied in alternating rounds:

• a **geometric step** (SurFree, Maho et al., CVPR 2021). With u the unit vector from the clean image to the current point at distance d, and v random and orthogonal to u, the point `z(t) = x + d·cos t·(cos t·u + sin t·v)` lies at distance `d·cos t`: a single accepted query shrinks the distance by cos t. Several values of t are tried per step in one batch and the largest accepted one is kept; the search angle adapts per image, and the radius is periodically re-bisected.

• a **random walk along the boundary** (Brendel, Rauber & Bethge, ICLR 2018): one orthogonal trial at constant distance plus one inward trial. The inward trial does not depend on the outcome of the orthogonal one, so both are answered in a single batched call, and both step sizes adapt per image to the observed acceptance rate.

Stage 4 — surviving 8-bit quantisation

The deliverable is a PNG, so the perturbation must survive rounding to the integer pixel grid. This is not a detail here: the converged perturbation is only a few levels per pixel, which is close to the quantisation step itself, and a boundary point rounds to the wrong side roughly half the time. Three candidate families are tried — deterministic rounding, rounding of slightly scaled-up perturbations, and several dithered (stochastic, unbiased) roundings — and the closest candidate the oracle confirms is kept.

Stage 5 — polishing on the integer grid

Three passes, all operating on integer pixel values and all verified by the oracle, so none can make the result worse: binary search for the smallest L_∞ clipping radius that still flips the label; binary search for the largest threshold below which perturbation components can be pruned to zero; and greedy group-wise shrinking of the remaining components by one level.

Stage 6 — verification that survives a change of machine

The refinement stages drive each perturbation onto the decision boundary, which leaves the top two logits separated by a numerically tiny amount. Three separate things were observed to flip the label back: the **batch size** (cuDNN selects its convolution kernel partly from the tensor shape, so an image can be classified differently in a batch of 13 and a batch of 1000); the **tensor layout** (a PIL / `ToTensor` loader returns a non-contiguous permuted view, which takes a different kernel than the contiguous tensor the attack computed); and **one unit in the last place** — dividing a uint8 tensor by 255 on the CPU and on the GPU are not bit-identical, a 6e-8 difference that is nevertheless enough.

Enumerating every combination a grader might use is hopeless, so the test is not “does it pass on many backends” but “does it pass *with room to spare*”: an image is accepted only if the perturbation **shrunk to 97 %** is still adversarial. A point that survives being pulled back towards the clean image is not sitting on the boundary, so a last-bit difference cannot argue it out of its label. That margin test is applied across five evaluators (three GPU batch sizes, two CPU batch sizes) and four input representations, including the exact one a PNG loader produces. Images without a margin are scaled up along their own direction in 5 % steps, with the whole set re-verified after each step so the batch shapes keep matching the final check.

The cost of insisting on this is small: 19 of the 1000 images needed scaling up, which moved the mean L2 from 58.99 to 59.01. Without it, 7 images that verified in memory were no longer adversarial once the files were read back from disk.

Finally the written files are read back, compared byte-for-byte with the verified tensors, and re-checked: 1000/1000 adversarial and 1000/1000 still adversarial at 97 % of the perturbation.

4. What the choices are worth

Measured on a fixed 40-image subset (4 per class) with a fixed seed, so the rows are directly comparable.

configuration	queries/img	L2 median	L2 mean	PSNR
query-only, HopSkipJump core				
10 bisection steps per re-projection	10,841	209.7	-	-
20 bisection steps per re-projection	10,841	134.3	-	-
probes in a 3x4x4 subspace	10,846	563.4	650.6	30.58
probes in a 3x8x8 subspace	10,843	246.8	285.5	36.67
probes in a 3x16x16 subspace	10,842	164.7	193.1	39.68
probes in the full 3x32x32 space	10,841	141.4	158.4	40.93
query-only, adding boundary refinement				
15 iterations, no refinement	4,506	179.8	208.8	38.75
15 it + random walk 1500	7,506	121.5	136.8	42.80
15 it + random walk 5000	14,506	76.3	84.2	46.53
15 it + geometric 2500	12,356	76.6	83.8	47.01
15 it + geometric 1700 + walk 2500	14,844	73.8	78.2	47.34
15 it + geometric 3000 + walk 3000	19,926	68.2	69.4	48.27
with the substitute-model stage, no HopSkipJump				
transfer directions only (20 images)	265	67.3	84.7	47.94
transfer + geometric 800 + walk 800 + grid 400	6,441	54.9	61.7	50.14
transfer + geometric 2000 + walk 2000 + grid 600	13,637	51.6	55.7	50.78
transfer + geometric 4000 + walk 4000 + grid 800	24,945	50.6	50.5	51.53

Distances are on the 0–255 pixel scale, lower is better; PSNR is a mean over images, higher is better. The two bisection-depth rows are medians before quantisation on a 20-image subset at iteration 35, the only pair measured that way; every other row is the full pipeline on the same 40 images. The substitute-model rows were measured without the cross-backend hardening of stage 6, which costs a few percent of distortion on the minority of images that need it.

5. Final attack parameters

parameter	value	meaning
`seeds`	`4`	clean other-class images bisected at initialisation
`bsearch`	`20`	bisection steps when re-projecting onto the boundary
`no_transfer`	`False`	True = pure query-only, no substitute model
`surrogate_epochs`	`20`	training epochs per substitute network
`iters`	`0`	HopSkipJump iterations
`b0`	`100`	probes in the first HopSkipJump iteration (grows as sqrt(t))
`bmax`	`400`	cap on probes per iteration
`refine_rounds`	`1`	alternating geometric / random-walk rounds
`geo`	`2000`	geometric steps per round (3 queries each)
`walk`	`2000`	random-walk steps per round (2 queries each)
`rescue_rounds`	`2`	extra refinement passes spent on the worst images only
`rescue_frac`	`0.3`	fraction of images counted as 'worst' in a rescue pass
`rescue_geo`	`700`	geometric steps per rescue pass
`rescue_walk`	`700`	random-walk steps per rescue pass
`grid_rounds`	`1`	refinement rounds run against the rounded (8-bit) objective

`grid_geo`	`800`	geometric steps per grid-space round
`grid_walk`	`800`	random-walk steps per grid-space round
`greedy_rounds`	`5`	greedy integer-shrink passes
`max_batch`	`2048`	images per forward pass (does not affect the attack)
`seed`	`0`	random seed

Inference budget by stage (cumulative, per image)

stage	seed_init	transfer	hsja	refine	finalize
cumulative queries / image	81	275	275	12,714	16,900

Total 16,899,931 inferences for 1000 images (16,900 per image). Verification queries are not included. Where a substitute model is used, its one-off labelling of the public CIFAR-10 split (60,000 queries, 60 per attacked image) is counted in the `transfer` column.